

INFO0027: Project 1 (20% – Groups of 2)

File duplication detector

Submission before 2026-04-17

1 File Deduplication

You will design a first-class data structure and associated algorithms to support the operations of an efficient file duplication detector. Files are said to duplicate each other if they are identical byte-by-byte.

Your job is to design an efficient ADT to identify sets of files (represented by their pathnames) whose contents are identical.

You must obviously avoid comparing all pairs of files individually, in order to achieve efficiency. One simple idea is to "summarise" the content of a file by a hash value and use that hash value to hone in on the necessary pair comparisons. An easy way to compute such hash value is to use tabulation hashing: you can construct a, say, 32 rows-by-256 columns table of 64-bit random numbers, and simply use the value of each byte in the file as a column index to retrieve the value to be XORed with the hash value, "rotating" through the rows of the table with the bytes of the file. Once you reach the end of the file, the hash value computed represents the file summary.

You should keep in mind that files that exhibit different sizes or different hash values cannot be identical.

2 FILEDEDUP ADT

More precisely, you will implement a FILEDEDUP ADT supporting the following operations:

1. FILEDEDUP FDInit(), that initializes the data structure and where FILEDEDUP is a pointer type identifying the created instance of the FILEDEDUP ADT;
2. `int` FDCheck(FILEDEDUP fd, `char` *filepath), which "adds" the content of the file whose path is given by the C-style string filepath to the file duplication detector; this function returns the value 1 on success, or 0 on error (such as if the file does not exist or is not accessible by the user).
3. `char` **FDDump(FILEDEDUP fd, `int` *length), which returns all the sets of identical files, identified by their pathnames, as an array of C-style strings, where each set is separated by a NULL pointer. The size of this array is communicated via the "return parameter" length, and you should also terminate the last set by a NULL pointer for simplicity.

The FILEDEDUP ADT will be used as a C library, from our point of view the data structure is opaque (`typedef struct magic *FILEDEDUP;`).

3 Evaluation

This project counts towards 20% of your final mark. In this project, you will be evaluated on:

- algorithms and data structures: implementation and motivation;

- code quality and maintainability: as in your code should be made for human to read not only for the computer to execute. Imagine a team handover, what should they know?
- Functional Suitability,
- Performance Efficiency,
- Test plan.

4 Deliverables

Projects must be submitted before 2026-04-17, 23:59, on Gradescope with the course entry code J6Y4N3. After this time, a penalty will be applied to late submissions. This penalty is calculated as a deduction of 2^{N-1} marks (where N is the number of started days after the deadline).

Your submission archive should at least contain your **implementation** of the FILEDEDUP ADT, as well as your **report** in PDF format, structured as showed in Figure 1:

```

teamName.zip
├── src
│   ├── filededup.c
│   └── filededup.h
└── *.pdf

```

Figure 1: Submission archive structure

The report should be in English, up to 3 pages long, explaining your ADT, algorithms and their complexity, and performance study. Your implementation will be compiled with `gcc`, with the following flags: `-Wall -pedantic -std=c11 -O3`, if you need something different contact us early.

Failure to compile will result in an awarded mark of 0. Likewise, poor spacial or time performance when run over large input will result in an awarded mark of 0. Warnings systematically result in lost marks.

Ask your questions early, and have fun...